

TEMA N° 2



TEMA 2 ARQUITECTURA Y COMPONENTES PARA EL DESARROLLO



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Estructurar proyectos móviles profesionales:** Identificando y organizando correctamente los directorios, archivos de configuración y dependencias esenciales para que tus aplicaciones sean escalables y fáciles de mantener.
2. **Dominar el comportamiento interno de una aplicación:** Comprendiendo el ciclo de vida de las pantallas (Actividades) y gestionando de forma segura y ética los permisos del dispositivo (como la cámara y el internet).
3. **Implementar patrones de arquitectura modernos:** Aplicando el patrón MVVM (Modelo-Vista-ViewModel) para separar el diseño visual de la lógica de programación, preparándote como un Técnico en Sistemas Informáticos altamente competitivo.



PRÁCTICA



¿Alguna vez te has preguntado por qué algunas aplicaciones móviles se cierran de la nada al girar la pantalla, o por qué agotan la batería y los megas de tu teléfono sin que te des cuenta?

Imagina el siguiente escenario: Has sido convocado en el nuevo y moderno módulo del CEA Herman Ortiz Camargo para un proyecto clave. Los comerciantes del Barrio Central Fátima y los productores que viven cerca del Barrio Saó (a solo 5 cuadras del CEA) necesitan una aplicación móvil para vender sus productos. La aplicación debe permitir a los usuarios ver un catálogo, usar la cámara para escanear códigos QR de descuentos en los negocios cercanos a la estación de tren (Barrio 24 de Septiembre), y geolocalizar entregas hacia áreas más alejadas como Ovidio Barberi o Las Misiones.

El equipo de estudiantes del semestre anterior intentó hacer la aplicación, pero se encontraron con graves problemas:

1. Al abrir la cámara en el Barrio Central Fátima, la aplicación se cerraba abruptamente porque nadie le dijo al sistema que necesitaban "permiso".
2. Al girar el celular de forma horizontal, toda la lista de productos descargados se borraba y volvía a consumir megas para descargarse de nuevo.
3. Todos los códigos de diseño, botones, conexión a internet y base de datos estaban mezclados en un solo archivo gigante e incomprensible.

Como Técnico en Sistemas Informáticos, tu misión de hoy no es escribir código a ciegas. Tu reto es pensar en la **arquitectura**. ¿Cómo organizamos esta casa antes de ponerle los muebles? ¿Cómo evitamos que la aplicación colapse cuando el usuario recibe una llamada mientras pide comida? Pensemos en las bases.



TEORÍA



2.1. ESTRUCTURA DE DIRECTORIOS DE UN PROYECTO MÓVIL

Cuando un Técnico en Sistemas Informáticos crea un proyecto nuevo (por ejemplo, en Android Studio), el sistema genera automáticamente una serie de carpetas y archivos. Entender dónde va cada cosa es el primer paso para no perderse.

1. **Carpeta manifests:** Contiene el archivo AndroidManifest.xml. Es la "cédula de identidad" de tu app.
2. **Carpeta java (o kotlin):** Aquí vive el cerebro de la aplicación. Contiene todo el código fuente (clases, funciones, lógica).
3. **Carpeta res (Recursos):** Almacena todo lo que no es código de programación:
 1. drawable: Imágenes, íconos y gráficos.
 2. layout: Los diseños visuales de las pantallas (archivos XML).
 3. values: Textos (strings), colores y estilos.
4. **Archivos Gradle Scripts:** Son los planos de construcción. Le dicen al sistema cómo ensamblar tu aplicación y qué herramientas externas utilizar.





2.2. COMPRENSIÓN DEL CICLO DE VIDA DE UNA ACTIVIDAD/PANTALLA

Una *Activity* (Actividad) es una sola pantalla en tu aplicación. Al igual que un ser humano, una Actividad nace, vive, se pausa y finalmente muere. Entender esto evitará el problema que tenían en el CEA de que la app se reiniciaba al girar la pantalla.

Los estados más importantes que debes controlar como Técnico son:

1. **onCreate():** La pantalla "nace". Aquí se enlaza el diseño visual con el código y se inicializan las variables. Solo ocurre una vez.
2. **onStart():** La pantalla se vuelve visible para el usuario.
3. **onResume():** El usuario ya puede interactuar (tocar botones, escribir). La app está en primer plano.
4. **onPause():** La pantalla pierde el foco (por ejemplo, entra una llamada o salta una notificación).
5. **onStop():** La pantalla ya no es visible (el usuario presionó el botón de inicio del celular).
6. **onDestroy():** La pantalla "muere" y es destruida por el sistema para liberar memoria RAM.





2.3. USO DEL ARCHIVO DE MANIFIESTO Y GESTIÓN DE PERMISOS (CÁMARA, INTERNET)

El archivo AndroidManifest.xml es un contrato entre tu aplicación y el sistema operativo del teléfono. Si tu app en Pailón necesita conectarse a internet para ver los eventos del coliseo en el Barrio 27 de Mayo, debes declararlo aquí. Si no lo haces, el sistema bloqueará la acción por seguridad.

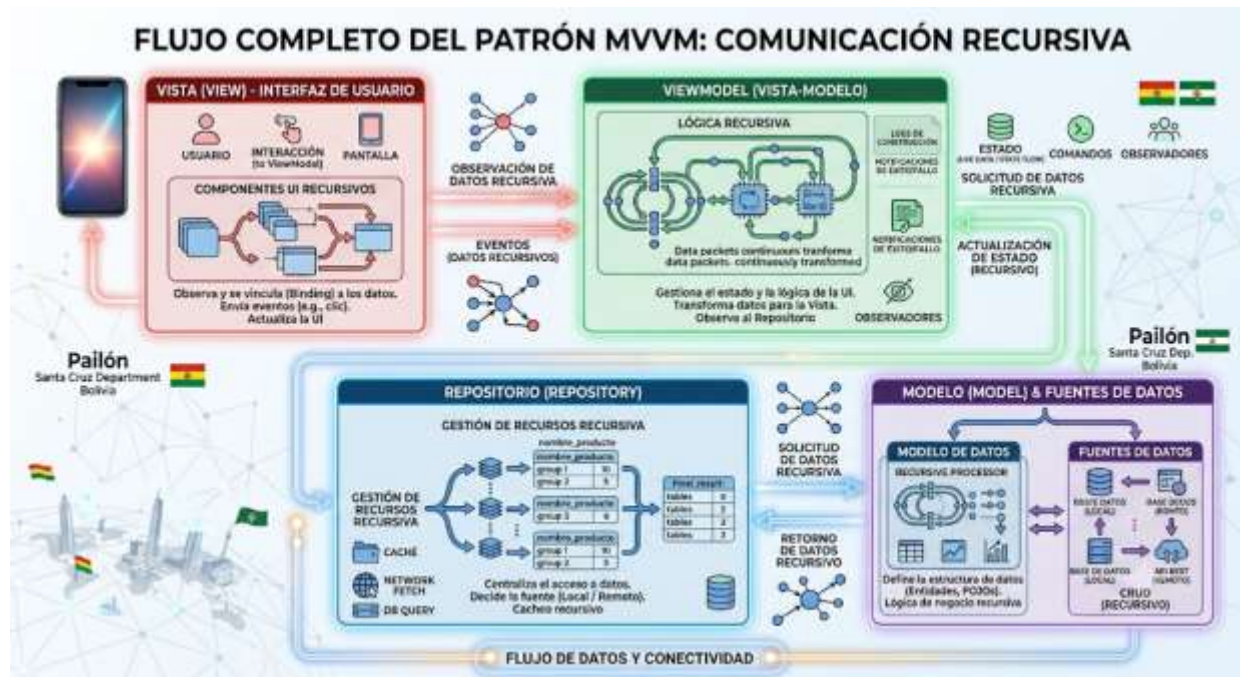
XML

```
<manifest xmlns:android="
http:
//schemas.android.com/apk/res/android"    package="
com.ceahermanortiz.pailonapp">    <uses-permission android:name="
android.permission.INTERNET" />    <uses-permission android:name="
android.permission.CAMERA" />    <application
android:icon="@mipmap/ic_launcher"    android:label="
Pailón App">    <activity android:name=".MainActivity">
<intent-filter>    <action android:name="
android.intent.action.MAIN" />    <category android:name="
android.intent.category.LAUNCHER" />    </intent-filter>
</activity>    </application>
</manifest>
```

2.4. PATRONES DE ARQUITECTURA RECOMENDADOS (MVVM BÁSICO)

Si pones toda la ropa, los platos y las herramientas en un solo cajón, nunca encontrarás nada. Lo mismo pasa en programación. Para evitar el código "espagueti", los profesionales usamos el patrón **MVVM (Model-View-ViewModel)**.

1. **Model (Modelo):** Es el gestor de datos. Representa la información (ej. la lista de negocios del Barrio Fátima o San Expedito) y decide si la obtiene de internet o de la base de datos interna.
2. **View (Vista):** Es la pantalla y los botones (lo que ve el usuario). Es "tonta", no toma decisiones lógicas, solo muestra información y detecta clics.
3. **ViewModel (Modelo de Vista):** Es el puente o traductor. Pide los datos al *Model*, los adapta y se los pasa a la *View*. Su gran ventaja es que **sobrevive a los giros de pantalla**, evitando que la información se pierda.



2.5. MANEJO DE DEPENDENCIAS Y LIBRERÍAS EXTERNAS

En lugar de inventar la rueda o programar desde cero cómo descargar una imagen de internet, un Técnico en Sistemas Informáticos eficiente usa "librerías" creadas por empresas como Google o Square. Estas se añaden en el archivo build.gradle (Module: app).

Groovy

```
// Archivo: build.gradle (Nivel de módulo)
dependencies {
// Librerías estándar de Android (ya incluidas)    implementation '
androidx.appcompat:appcompat:1.6.1
'           // DEPENDENCIAS EXTERNAS AGREGADAS POR EL TÉCNICO:    // 1. Retrofit:
Excelente para conectar la app a Internet (APIs)    implementation '
com.squareup.retrofit2:retrofit:2.9.0
'           // 2. Glide: Descarga y muestra imágenes eficientemente
implementation '
com.github.bumptech.glide:glide:4.15.1'}
```

2.6. ASISTENTES DE INTELIGENCIA ARTIFICIAL EN EL DESARROLLO (TENDENCIA)

El desarrollo móvil moderno ha integrado profundamente la Inteligencia Artificial (IA). Herramientas como *Gemini en Android Studio* o *GitHub Copilot* actúan como un co-piloto para el Técnico. No hacen el trabajo por ti, pero aceleran tareas repetitivas. Por ejemplo, puedes



pedirle a la IA: "Genera la estructura básica de un ViewModel para listar los parques del Barrio Residencial Norte", y la IA creará el código base o plantilla, dejándote a ti la tarea de afinar la lógica arquitectónica.

2.7. ARQUITECTURA REACTIVA Y FLUJOS ASÍNCRONOS

En poblaciones donde la conectividad puede fluctuar (por ejemplo, yendo hacia los Jardines de Pailón o María Belén), no podemos permitir que la aplicación se "congele" esperando datos. La arquitectura moderna utiliza un enfoque *Reactivo* mediante **Corrutinas y Flujos (Flows)** en Kotlin. Esto permite que la aplicación trabaje en un "hilo secundario" (en el fondo) descargando datos sin bloquear la pantalla principal, haciendo que la experiencia del usuario sea fluida y profesional.

2.8. INYECCIÓN DE DEPENDENCIAS (DI) MODERNA

En proyectos grandes, crear manualmente los objetos de conexión a la base de datos o a internet consume mucha memoria y genera errores. Hoy en día se utilizan librerías como **Hilt** o **Dagger**. La Inyección de Dependencias es como tener un almacén central en tu código que automáticamente entrega las herramientas (clases) exactas a las pantallas que las necesitan, sin que la pantalla tenga que fabricarlas por sí misma.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito Común	Realidad Técnica
"Un buen Técnico escribe todo el código desde cero."	FALSO. Un buen Técnico utiliza arquitecturas modernas (MVVM) y librerías probadas (dependencias) para ahorrar tiempo, reducir errores y asegurar calidad.
"Los permisos de la cámara e internet son automáticos."	FALSO. Por seguridad y privacidad del usuario, todo permiso sensible debe declararse obligatoriamente en el archivo AndroidManifest.xml.



□ *Glosario de Términos*

1. **Activity (Actividad):** Componente central de una app en Android que proporciona una ventana o pantalla con la que los usuarios pueden interactuar.
2. **Gradle:** Herramienta de automatización de compilación utilizada en Android para gestionar las dependencias y empaquetar la aplicación final.
3. **AndroidManifest.xml:** Archivo raíz de todo proyecto móvil que describe la información fundamental de la app al sistema operativo.
4. **Dependencia:** Librería o paquete de código de terceros que se integra al proyecto para añadir funcionalidades específicas (ej. Glide, Retrofit).
5. **MVVM:** Patrón de arquitectura de software que separa la interfaz de usuario (View), la lógica de presentación (ViewModel) y la gestión de datos (Model).

□ *Ponte a Prueba*

1. **¿Qué método del ciclo de vida se ejecuta cuando una Actividad ya no es visible para el usuario (por ejemplo, presiona el botón Home)?**

- a) onCreate()
- b) onStop()
- c) onResume()

2. **¿En qué archivo de la estructura del proyecto debemos colocar las imágenes y los diseños de pantalla (layouts)?**

- a) En la carpeta java
- b) En la carpeta manifests
- c) En la carpeta res

3. **En el patrón MVVM, ¿cuál es la función principal del ViewModel?**

- a) Guardar información permanentemente en el disco duro.



b) Ser el puente entre los datos (Modelo) y la pantalla (Vista), sobreviviendo a los giros del teléfono.

c) Diseñar los colores de los botones.

(Respuestas: 1-b, 2-c, 3-b)



VALORACIÓN



Como futuros profesionales del CEA Herman Ortiz Camargo, desarrollar aplicaciones móviles no es solo escribir código, es asumir una responsabilidad social y medioambiental.

Piensa en los habitantes de Pailón, desde el centro en el Barrio 24 de Septiembre hasta San José o Oto Waver. No todos tienen el teléfono celular de última generación. Si no aplicamos una buena arquitectura, como optimizar el ciclo de vida de nuestra Actividad o manejar bien las dependencias, crearemos aplicaciones pesadas que consumirán rápidamente la batería y saturarán la memoria RAM. Esto obliga a las personas a pensar que su teléfono "ya no sirve" y desecharlo, generando **e-waste (basura electrónica)** que contamina nuestra región.

Además, el uso del archivo AndroidManifest.xml nos enfrenta a la **ética de la privacidad**. Si creamos una app para los comerciantes de Central Fátima, y pedimos permiso a la cámara, el micrófono o el GPS, debemos hacerlo única y exclusivamente para las funciones que la aplicación requiere. Solicitar permisos innecesarios vulnera la seguridad del usuario. Un buen Técnico en Sistemas Informáticos protege la confianza de su comunidad desarrollando software eficiente, limpio, ético y transparente.



PRODUCCIÓN



Reto Final: Laboratorio "Estructura Base - Pailón Market App"

Aplica lo aprendido en los equipos de computación del CEA. Sigue este paso a paso para configurar la arquitectura inicial de una aplicación.

**Paso 1: Exploración del Directorio**

1. Abre Android Studio y crea un nuevo proyecto seleccionando "Empty Views Activity" (o actividad vacía).
2. Nómbralo PailonMarket.
3. Navega por el árbol de directorios a la izquierda. Ubica y abre la carpeta app -> src -> main.

Paso 2: Configuración de Permisos (Manifiesto)

1. Abre el archivo AndroidManifest.xml.
2. Justo encima de la etiqueta <application>, escribe el código necesario para solicitar el permiso de Internet:

XML

```
<uses-permission android:name="
android.permission.INTERNET" />
```

Paso 3: Estructurando MVVM

1. Ve a tu carpeta java (donde está tu paquete principal com.tu_nombre.pailonmarket).
2. Haz clic derecho sobre la carpeta principal -> New -> Package.
3. Crea tres paquetes nuevos (carpetas) llamados: model, view, y viewmodel.
4. Arrastra tu archivo MainActivity.kt (o .java) dentro de la carpeta view. ¡Acabas de organizar tu primer proyecto usando los principios de MVVM!

Paso 4: Añadiendo una Dependencia

1. En el panel izquierdo, busca la sección "Gradle Scripts" y abre build.gradle.kts (Module :app).
2. En el bloque de dependencies { ... }, añade la siguiente línea para incluir una librería externa de imágenes (Glide):



Kotlin

```
// Dependencia para cargar fotos de Los productos de Pailónimplementation("com.github.bumptech.glide:glide:4.16.0")
```

1. Haz clic en el botón flotante "Sync Now" que aparecerá en la parte superior derecha para que el sistema descargue la librería.

¡Felicidades, Técnico! Has construido las bases sólidas de una aplicación profesional lista para servir a la comunidad de Pailón.

5. RECURSOS ADICIONALES

Para profundizar en tu aprendizaje y mejorar tus habilidades como Técnico en Sistemas Informáticos, te recomiendo explorar los siguientes enlaces oficiales y herramientas:

1. **Documentación Oficial de Android (Arquitectura):** La guía principal desarrollada por Google para entender las mejores prácticas y el patrón MVVM a fondo. *Enlace sugerido de búsqueda: Guía de Arquitectura de apps de Android Developer.*
2. **Android Studio GitHub Samples:** Un repositorio oficial lleno de ejemplos prácticos y proyectos reales creados por expertos, listos para que revises su código fuente. *Enlace sugerido de búsqueda: Android Architecture Blueprints en GitHub.*
3. **Material Design 3:** Aunque te enfoques en la arquitectura, conocer la guía oficial de diseño de Google te ayudará a estructurar mejor tus archivos dentro de la carpeta res (colores, tipografías y formas). *Enlace sugerido de búsqueda: Material Design para Android.*